

Trabajo Practico: Agentes para Catan con Heurísticas y Modelos de Lenguaje

[Nombre 1] [Nombre 2]

21 de abril de 2026

1. Objetivo y alcance

Este trabajo desarrolla y evalúa un agente inteligente para Catan sobre el simulador PyCatan, combinando dos enfoques:

- un agente heurístico basado en reglas y funciones de evaluación;
- una extensión con modelos de lenguaje (LLM) para decisiones concretas del juego.

El objetivo no es solo maximizar victorias, sino analizar de forma crítica la viabilidad real de usar LLM en un entorno multiagente con restricciones de latencia, robustez y coste computacional.

2. Parte 1: Desarrollo del agente heurístico

2.1. Diseño general

Se implementó el agente `AlexPelochojaimHeuristicAgent` con una política orientada a:

- maximizar producción esperada de recursos (ponderada por probabilidad de dados);
- mantener diversidad de recursos para evitar cuellos de botella;
- favorecer expansión y acceso a nodos de alto valor;
- estabilizar progreso con comercio con banca dirigido por objetivo de construcción.

2.2. Decisiones controladas por el agente

1. **Colocación inicial** (`on_game_start`): selección de nodo y carretera inicial en función de puntuación de asentamiento y potencial de expansión.
2. **Fase de construcción** (`on_build_phase`): prioridad ciudad > pueblo > carretera > carta de desarrollo.
3. **Fase de comercio** (`on_commerce_phase`): objetivo dinámico de construcción y selección de intercambio con banca para cubrir faltantes.
4. **Cartas y eventos**: uso de monopolio, year of plenty, road building y movimiento del ladrón con reglas de valor esperado.

2.3. Informacion del estado usada para decidir

- recursos disponibles y faltantes por coste de construccion;
- nodos y carreteras legales;
- probabilidades de produccion por terrenos adyacentes;
- puertos y ratios de intercambio;
- ocupacion propia/rival de nodos para priorizar objetivos.

2.4. Justificacion de las heurísticas

La combinacion de produccion + diversidad + expansion reduce el riesgo de bloqueos por escasez de recursos y mantiene ritmo de construccion. La prioridad de ciudad y pueblo se alinea con impacto directo en puntos de victoria y economia de largo plazo.

3. Parte 1.2: Evaluacion mediante benchmarks

3.1. Configuracion experimental

Se han utilizado los benchmarks del proyecto:

- `benchmark_vs_random.py`
- `benchmark_vs_agentes_estandar.py`

En esta iteracion se ejecutaron corridas ligeras para validar integracion extremo a extremo con Ollama local y recoger trazas de fallback.

3.2. Resultados disponibles

3.2.1. Benchmark contra random (corrida ligera actual)

Fuente: `benchmark_vs_random_resultados.csv`

Agente	Partidas	Victorias	Win rate	Media puntos	Decisiones LLM	Fallback rate
Heuristico	4	1	0.2500	2.00	0	0.0000
LLM (qwen2.5:0.5b)	4	0	0.0000	2.00	15	1.0000

Cuadro 1: Resultado actual frente a random.

3.2.2. Benchmark contra estandar (corrida ligera actual)

Fuente: `benchmark_vs_estandar_resultados.csv`

Agente	Partidas	Victorias	Win rate	Media puntos	Decisiones LLM	Fallback rate
Heuristico	4	2	0.5000	2.25	0	0.0000
LLM (qwen2.5:0.5b)	4	1	0.2500	2.25	15	1.0000

Cuadro 2: Resultado actual frente a agentes estandar.

3.3. Analisis breve

- Ambas corridas se ejecutaron con presupuesto muy reducido (`max_rounds=1`), por lo que no deben interpretarse como ranking final de rendimiento.
- En este escenario rapido, el agente heuristico mantiene mejor rendimiento relativo que la version LLM.
- El agente LLM registra decisiones, pero en esta configuracion la tasa de fallback es total.
- Por ello, el rendimiento observado del agente LLM queda fuertemente contaminado por politica heuristica de respaldo.

4. Parte 2: Uso de modelos de lenguaje en el agente

4.1. Diseno de la interaccion con el LLM

La interfaz LLM se diseno para minimizar ambigüedad y tokens:

- estado de partida serializado en JSON reducido;
- lista cerrada de acciones legales;
- salida exigida como JSON con `action_index`.

Se probaron tres variantes de prompt (`compact_json`, `resource_focus`, `safe_legal`).

4.2. Integracion en el agente

El LLM se integra en dos decisiones requeridas por el enunciado:

1. `on_game_start`;
2. `on_build_phase` (decision frecuente).

Si hay error de proveedor, timeout, salida invalida o indice fuera de rango, el sistema aplica fallback a la decision heuristica, garantizando robustez funcional.

4.3. Modelos y entornos evaluados

En este momento hay resultados instrumentados con:

- **Ollama local:** `qwen2.5:0.5b`

Pendiente para completar al 100 % los requisitos de la practica:

- al menos un modelo en AWS Bedrock;
- al menos un modelo via API UPV;
- comparativa de al menos 3 modelos en total.

4.4. Observaciones tecnicas importantes

Durante la ejecucion en entorno mixto WSL + PowerShell se observaron dos efectos clave:

- desde WSL no siempre hay acceso a la API HTTP local de Ollama en Windows;
- el fallback por CLI puede incrementar latencia de forma significativa y afectar estabilidad de benchmarks largos.

Este comportamiento explica parte de las ejecuciones incompletas o muy lentas y el aumento de fallback efectivo.

5. Comparacion de codigo: Shiyi_Catan vs PyCatan/Agents

Se compararon los archivos homonimos en ambos directorios.

5.1. Resumen

- `AlexPelochJaimeHeuristicAgent.py`: misma logica funcional.
- `AlexPelochJaimeLLMAgent.py`: misma logica funcional.
- `heuristic_core.py`: misma logica funcional.
- `llm_prompts.py`: mismo contenido funcional.
- `llm_engine.py`: **si presenta diferencias funcionales relevantes** en PyCatan/Agents.

5.2. Diferencias funcionales relevantes en `llm_engine`

Respecto a `Shiyi_Catan/llm_engine.py`, la version de `PyCatan/Agents/llm_engine.py` incluye:

- fallback por CLI de Ollama cuando falla la API HTTP;
- resolucion de rutas candidatas para `ollama.exe` (incluyendo `CATAN_OLLAMA_CLI_PATH`);
- forzado de respuesta JSON y temperatura 0 en payload de Ollama;
- parser mas robusto de `action_index` (JSON estricto + extraccion relajada por regex).

Estas mejoras incrementan tolerancia a fallos de conectividad/formato, pero tambien pueden introducir mayor latencia cuando se activa transporte por CLI.

6. Uso de herramientas de IA en el desarrollo

Se utilizaron herramientas de IA generativa (principalmente GitHub Copilot Chat) para:

- acelerar refactor y estructuracion de agentes;
- instrumentar benchmark y metricas de fallback;
- diagnosticar conectividad Ollama y causas de degradacion.

Utilidad percibida: alta para iteracion y depuracion.

Limitaciones: necesidad de validacion manual de resultados experimentales y control estricto de reproducibilidad.

7. Conclusiones

1. El agente heuristico presenta un comportamiento consistente y competitivo para el objetivo de la practica.
2. La integracion LLM esta implementada en dos decisiones clave, con mecanismo de seguridad correcto.
3. En las corridas actuales, la tasa de fallback del LLM es elevada, por lo que la evaluacion de calidad intrinseca del modelo todavia es incompleta.
4. Para cierre definitivo se debe completar la matriz de comparacion (prompts y modelos) incluyendo Bedrock y UPV bajo un protocolo experimental uniforme.

8. Trabajo futuro inmediato

- ejecutar benchmarks ligeros comparables por modelo/prompt con control de latencia;
- anadir tabla de tiempos medios por decision y, cuando sea posible, tokens entrada/salida;
- consolidar resultados en una figura comparativa final (win rate, fallback rate, latencia);
- ajustar redaccion final para no superar 10 paginas.